INTRODUCTION TO DATA SCIENCE · AUDITORY EXERCISES

Exercise 3 · Part 1: Data and Classical Models

Data Preparation

Ana Todorovska
Dimitar Peshevski
Marija Stojcheva
Viktor Kostadinoski

Faculty of Computer Science and Engineering
Ss. Cyril and Methodius University in Skopje

Winter Semester
2026/2027

Loading the Data

PYTHON

```python
import numpy as np
import pandas as pd
from scipy import stats
from sklearn import compose, ensemble, impute, neighbors, preprocessing
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.experimental import enable_iterative_imputer  # noqa: F401
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline

SEED = 2026
rng = np.random.default_rng(SEED)
```

The file marks a missing value with `?`, so `read_csv` takes `na_values="?"`. Without it, every column that holds a `?` would be read as text.

The Worked Example



The Automobile data list 205 cars imported into the United States in 1985, with 25 columns that describe each car and its list price in dollars. The session turns the raw table into a table ready for modelling, and every decision in it has a reason that the data support.

01

Missing Values

Discovery, mechanisms, deletion, imputation, and interpolation

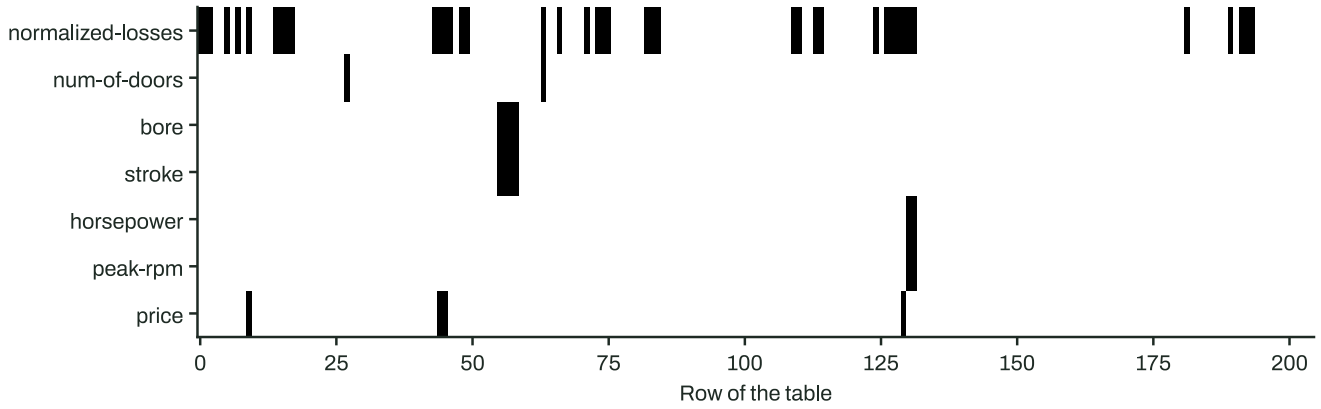
The Cars


```python
cars[["make", "body-style", "num-of-doors", "horsepower", "normalized-losses", "price"]].head(4)
```

	make	body-style	num-of-doors	horsepower	normalized-losses	price
0	alfa-romero	convertible	two	111.0	NaN	13495.0
1	alfa-romero	convertible	two	111.0	NaN	16500.0
2	alfa-romero	hatchback	two	154.0	NaN	16500.0
3	audi	sedan	four	102.0	164.0	13950.0

The columns are measurements, counts written as words, labels, and a `make` with 22 values. `normalized-losses` is the average insurance loss per insured vehicle year.

Where the Gaps Are



PYTHON

```
missing = cars.isna().sum()
missing[missing > 0]
```

OUTPUT

```
normalized-losses    41
num-of-doors          2
bore                  4
stroke                4
horsepower            2
peak-rpm              2
price                 4
dtype: int64
```

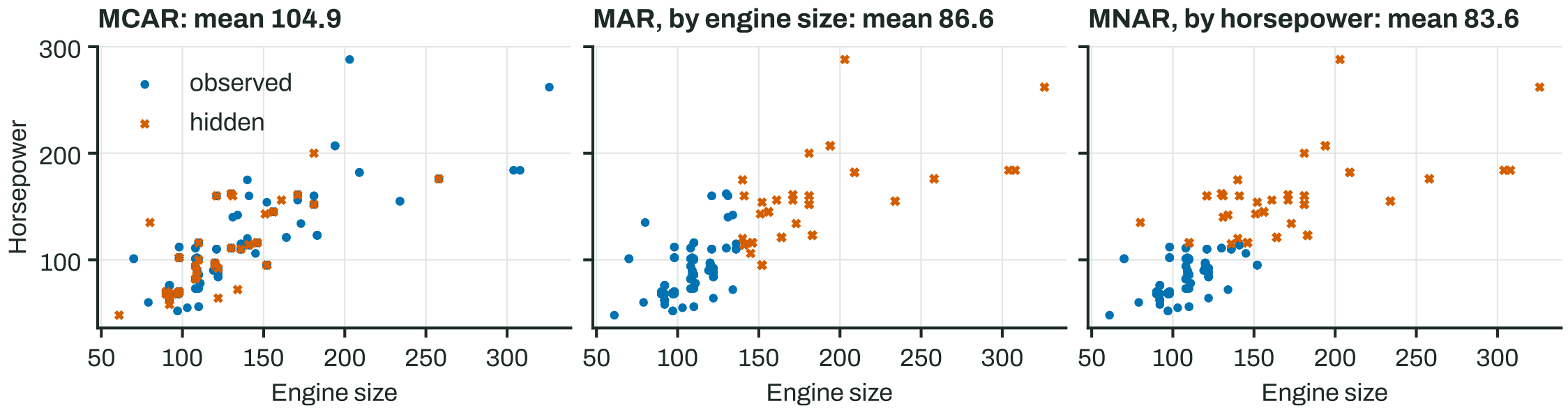
The four cars without `bore` and `stroke` have a rotary engine, which has no cylinders, so the value does not exist. The gaps in `normalized-losses` cluster by make.

Three Mechanisms of Missingness

Mechanism	The Chance of a Gap Depends on	Consequence
Missing completely at random (MCAR)	Nothing.	The observed values are a fair sample, and any imputation is unbiased on average.
Missing at random (MAR)	Other observed columns.	A model that uses those columns can fill the gap.
Missing not at random (MNAR)	The missing value itself.	No observed column fully explains the gap, and every imputation is biased.

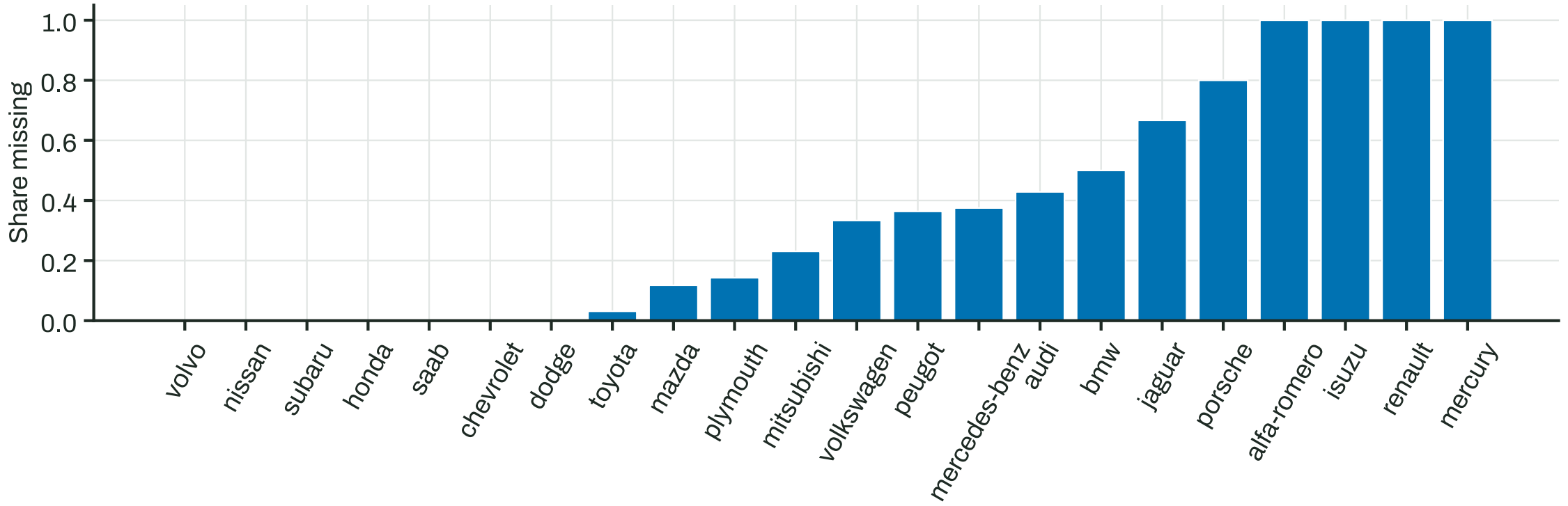
The mechanism is a property of how the data were collected, so the table alone can rule out MCAR but cannot prove MAR.

What Each Mechanism Does to a Mean



The horsepower of 30 % of the cars is hidden three ways. Under MCAR the observed mean stays near the true mean of 104.3. Under MAR and MNAR the powerful cars go missing and the mean falls, but under MAR the observed engine size still predicts the hidden values.

Missing Losses by Make



The losses are missing for every car of four makes and for none of seven others, so they are not missing completely at random. The gap depends on the make, which a missingness flag and an imputer can use.

Dropping Rows or Columns

PYTHON

```python
complete_rows = cars.dropna()
complete_columns = cars.dropna(axis=1)
len(complete_rows), sorted(set(cars["make"]) - set(complete_rows["make"])), complete_columns.shape[1]
```

OUTPUT

```
(159, ['alfa-romero', 'isuzu', 'mercury', 'renault'], 19)
```

Dropping every row with a gap keeps 159 cars, and four makes disappear entirely, so the rest are no longer a fair sample. Dropping every column with a gap loses seven columns, the price among them. Only the rows without a price are dropped, because a car without the response can neither train nor test a model.

The Split Comes First

PYTHON

```python
cars = cars.dropna(subset=["price"]).reset_index(drop=True)
X, y = cars.drop(columns="price"), cars["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=SEED)
len(X_train), len(X_test)
```

OUTPUT

```
(150, 51)
```

Every later step learns a statistic from the data, so the test rows are set aside before any of them runs. `random_state` fixes the shuffle, so the split is the same on every run.

Simple Imputation

PYTHON

```
numeric = ["symboling", "normalized-losses", "wheel-base", "length", "width", "height", "curb-weight",
           "engine-size", "bore", "stroke", "compression-ratio", "horsepower", "peak-rpm", "city-mpg",
           "highway-mpg"]
median = impute.SimpleImputer(strategy="median").fit(X_train[numeric])
pd.Series(median.statistics_, index=numeric)[["normalized-losses", "bore", "horsepower"]]
```

OUTPUT

```
normalized-losses    113.00
bore                   3.31
horsepower            95.00
dtype: float64
```

`SimpleImputer` learns one value per column from the training rows, here the median, and fills every gap of that column with it, in the training rows and in the test rows alike.

Model-Based Imputation

PYTHON

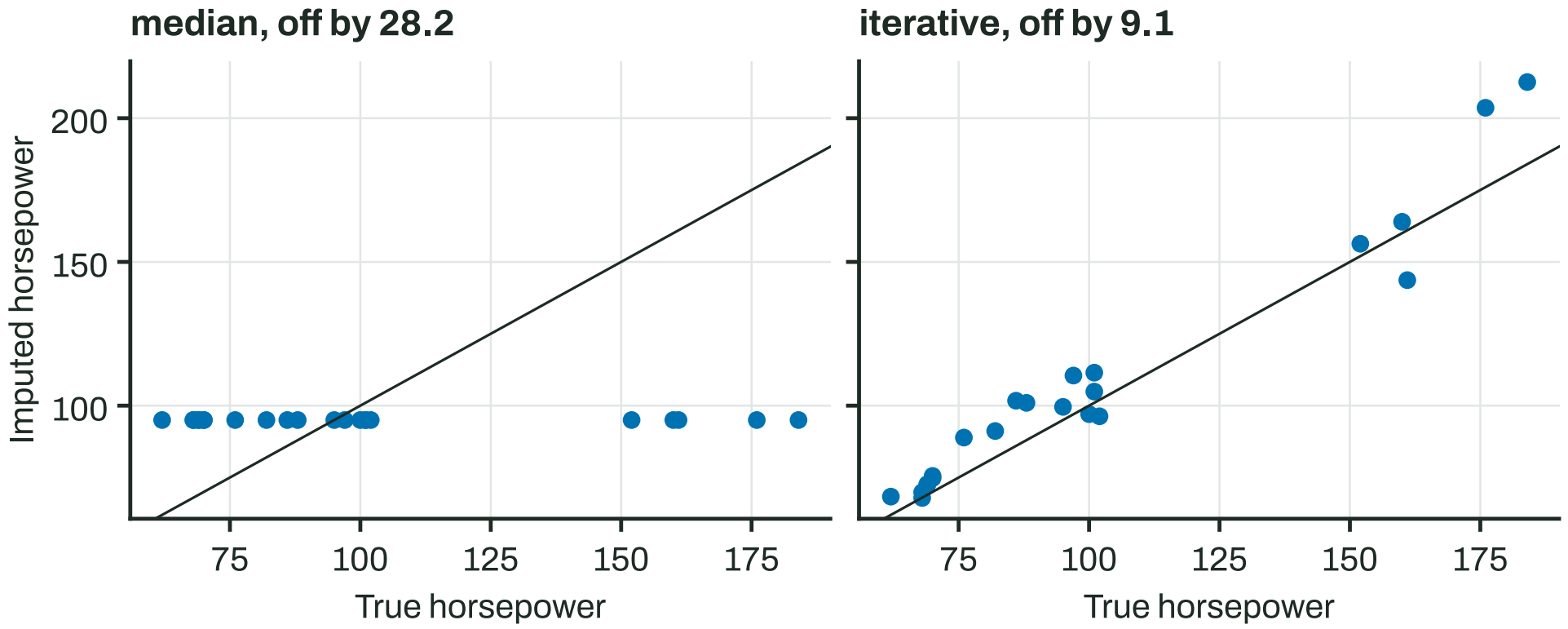
```python
hp = "horsepower"
known = X_train[numeric].dropna(subset=[hp])
mask = np.random.default_rng(SEED).random(len(known)) < 0.2
masked = known.copy()
masked.loc[mask, hp] = np.nan
filled = impute.IterativeImputer(max_iter=20, random_state=SEED).fit_transform(masked)
round(np.abs(known.loc[mask, hp].to_numpy() - filled[mask, numeric.index(hp)]).mean(), 1)
```

OUTPUT

```
np.float64(9.1)
```

A fifth of the known horsepower values are hidden and imputed, so the imputation can be compared with the truth. `IterativeImputer` predicts each column from the others, in rounds, and `KNNImputer` averages the nearest complete rows.

Imputers Compared



The median gives every hidden car the same value and misses by about 28 horsepower on average. The engine size and the weight predict horsepower well, so the iterative imputer misses by about 9.

Missingness Indicators

PYTHON

```
indicator = impute.MissingIndicator().fit(X_train[numeric])
indicator.get_feature_names_out()
```

OUTPUT

```
array(['missingindicator_normalized-losses', 'missingindicator_bore',
       'missingindicator_stroke'], dtype=object)
```

A filled value hides that it was missing, and the absence can carry information, as it does for the losses of rare makes. `MissingIndicator` adds a flag column for each column that had gaps during fitting, and `add_indicator=True` makes an imputer append the same flags.

Interpolation

PYTHON

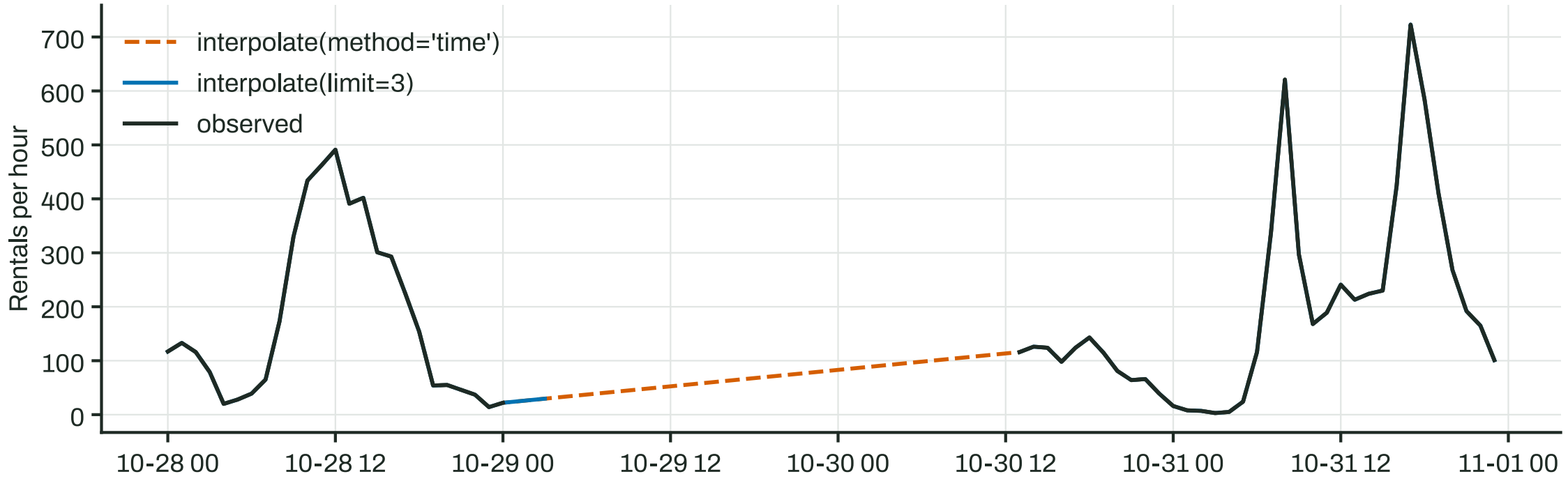
```
bikes = pd.read_csv("data/bike-hours.csv")
when = pd.to_datetime(bikes["date"]) + pd.to_timedelta(bikes["hour"], unit="h")
rentals = bikes.set_index(when)["rentals"].asfreq("h")
short = rentals.interpolate(method="time", limit=3, limit_area="inside")
rentals.isna().sum(), short.isna().sum()
```

OUTPUT

```
(np.int64(165), np.int64(74))
```

`asfreq("h")` inserts a row for each of the 165 missing hours of the bike rentals of Exercise 2, and interpolation fills a gap from its neighbours in time. Most gaps fall at night, where a missing row most likely means no rentals, so `fillna(0)` is the other reasonable fill.

Interpolation Across the Storm



The storm Sandy closed the bike system for 36 hours on 29 and 30 October 2012. A straight line across the gap invents rentals for a closed system, and `limit=3` fills only the first hours of a gap and leaves the rest missing.

NOTEBOOK · SECTIONS 1 TO 7

The Table and Its Gaps

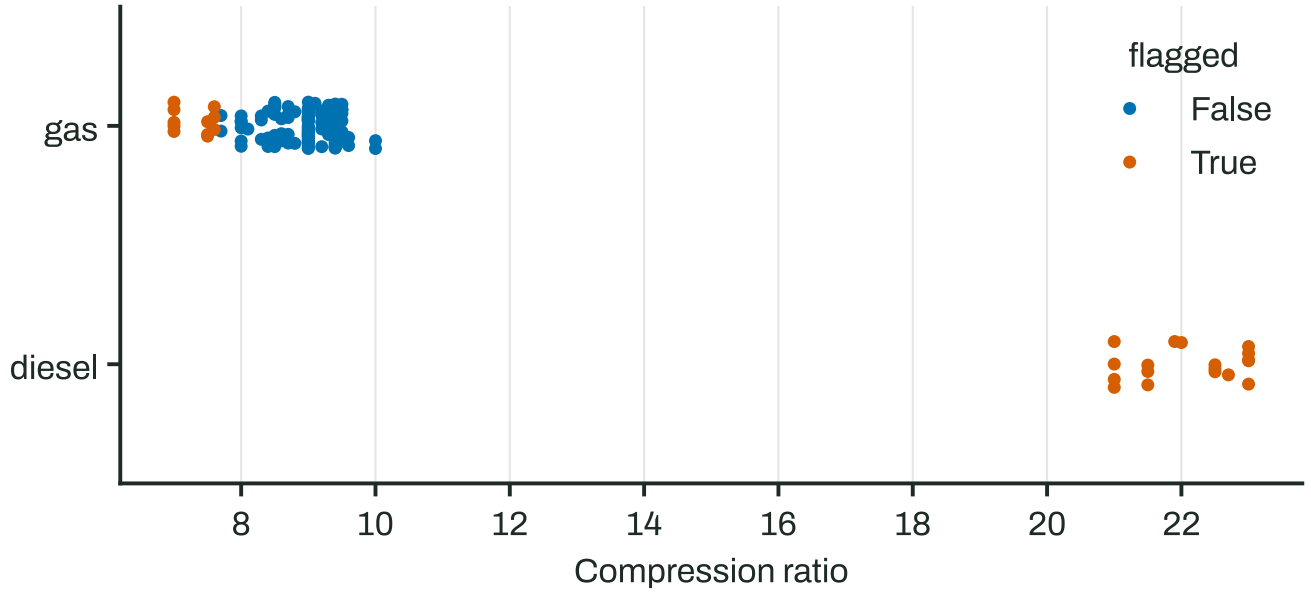
Run the cells from `## Setup` to `## 7. Interpolation`.

02

Outliers, Encoding, and Scaling

Detecting and handling outliers, turning labels into numbers, and comparable scales

Two Rules for Outliers



PYTHON

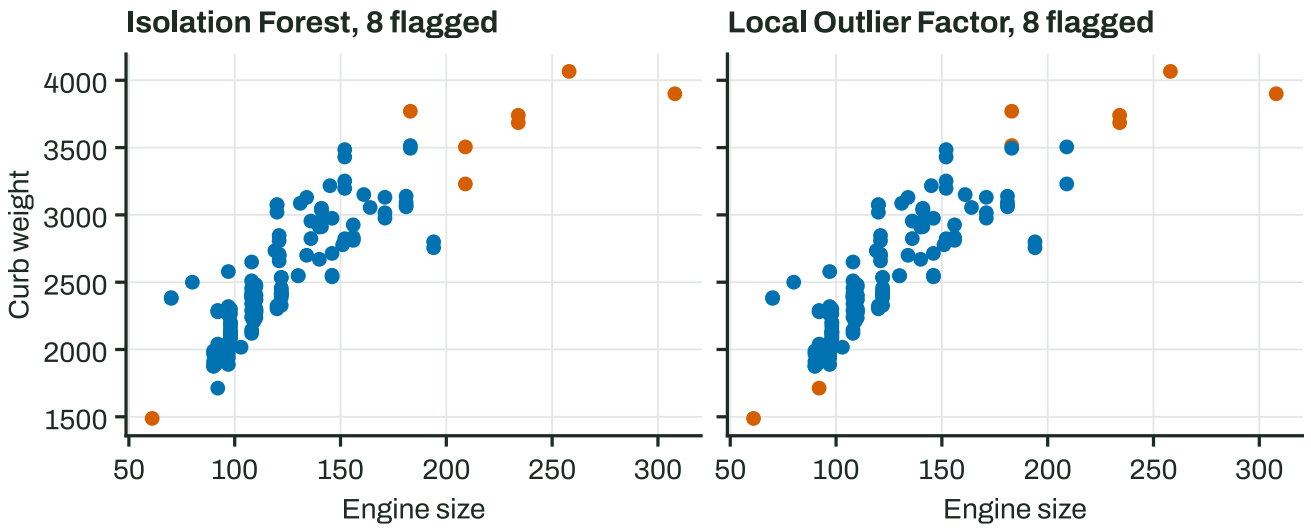
```
ratio = X_train["compression-ratio"]
q1, q3 = ratio.quantile([0.25, 0.75])
iqr = q3 - q1
flagged = ((ratio < q1 - 1.5 * iqr)
           | (ratio > q3 + 1.5 * iqr))
z = np.abs(stats.zscore(ratio))
flagged.sum(), (z > 3).sum()
```

OUTPUT

```
(np.int64(29), np.int64(0))
```

The IQR rule flags every diesel car, because a diesel engine compresses its air about twice as much. The values are correct, so they stay. The z-score flags none, because the diesel cars inflate the standard deviation it divides by.

Outliers Among Several Columns



PYTHON

```
pair = X_train[["engine-size",
                "curb-weight"]]
forest = ensemble.IsolationForest(
    contamination=0.05,
    random_state=SEED).fit_predict(pair)
lof = neighbors.LocalOutlierFactor(
    n_neighbors=20,
    contamination=0.05).fit_predict(pair)
(forest == -1).sum(), (lof == -1).sum()
```

OUTPUT

```
(np.int64(8), np.int64(8))
```

Isolation Forest flags the rows that random splits isolate quickly, and Local Outlier Factor flags the rows whose neighbourhood is sparser than that of their neighbours. Both return -1 for an outlier.

Keep, Cap, Transform, or Remove

PYTHON

```python
low, high = X_train["engine-size"].quantile([0.01, 0.99])
capped = X_test["engine-size"].clip(low, high)
(X_test["engine-size"] != capped).sum()
```

OUTPUT

```
np.int64(3)
```

An outlier is removed only when it is an error. A real but extreme value is kept, capped at bounds learned from the training rows, or transformed so that it no longer dominates. The flagged cars here are real luxury cars, so they are kept.

Three Kinds of Category

Kind	Example	Encoding
Ordinal, with an order	`num-of-cylinders`	`OrdinalEncoder` with the categories in their order.
Nominal, without an order	`body-style`	`OneHotEncoder`, one 0 or 1 column per category.
Nominal with many rare categories	`make`, 22 values	Frequency encoding, the share of the training rows.

An ordinal code keeps the order but not the distances, and a one-hot encoding of a variable with many rare categories adds many columns that each hold only a few cars.

Ordinal and One-Hot Encoding

PYTHON

```
ordinal = ["num-of-doors", "num-of-cylinders"]
orders = [["two", "four"], ["two", "three", "four", "five", "six", "eight", "twelve"]]
nominal = ["fuel-type", "aspiration", "body-style", "drive-wheels", "engine-location", "engine-type",
           "fuel-system"]
one_hot = preprocessing.OneHotEncoder(drop="if_binary", handle_unknown="ignore", sparse_output=False)
one_hot.fit(X_train[nominal]).get_feature_names_out()[:4]
```

OUTPUT

```
array(['fuel-type_gas', 'aspiration_turbo', 'body-style_convertible',
       'body-style_hardtop'], dtype=object)
```

Without `categories=`, `OrdinalEncoder` sorts the words alphabetically. `drop="if_binary"` keeps one column for a variable with two categories, and `handle_unknown="ignore"` encodes a new category as zeros.

Frequency Encoding

PYTHON

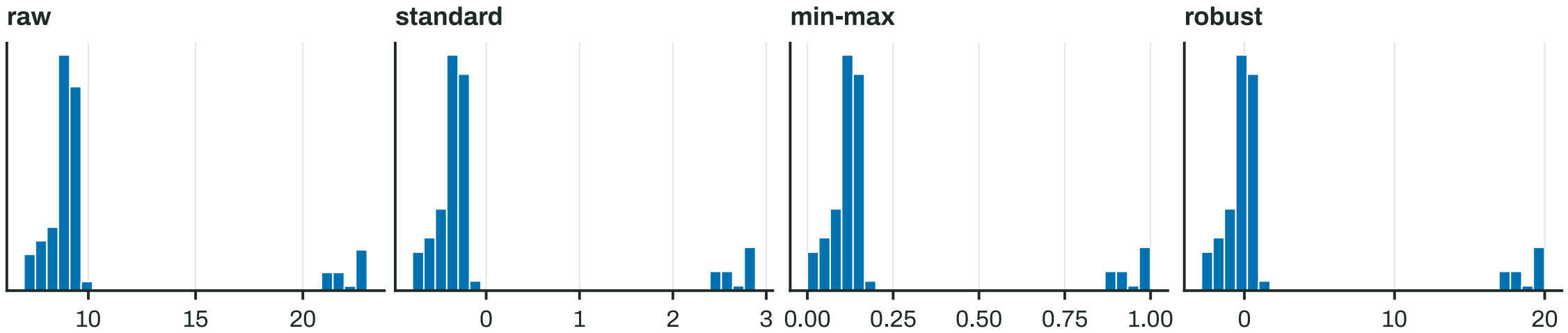
```python
class FrequencyEncoder(TransformerMixin, BaseEstimator):
    def fit(self, X: pd.DataFrame, y: object = None) -> "FrequencyEncoder":
        self.shares_ = {c: X[c].value_counts(normalize=True) for c in X.columns}
        return self

    def transform(self, X: pd.DataFrame) -> np.ndarray:
        return np.column_stack([X[c].map(s).fillna(0).to_numpy(dtype=float) for c, s in self.shares_.items()])

    def get_feature_names_out(self, input_features: object = None) -> np.ndarray:
        return np.array([f"{c}_frequency" for c in self.shares_])
```

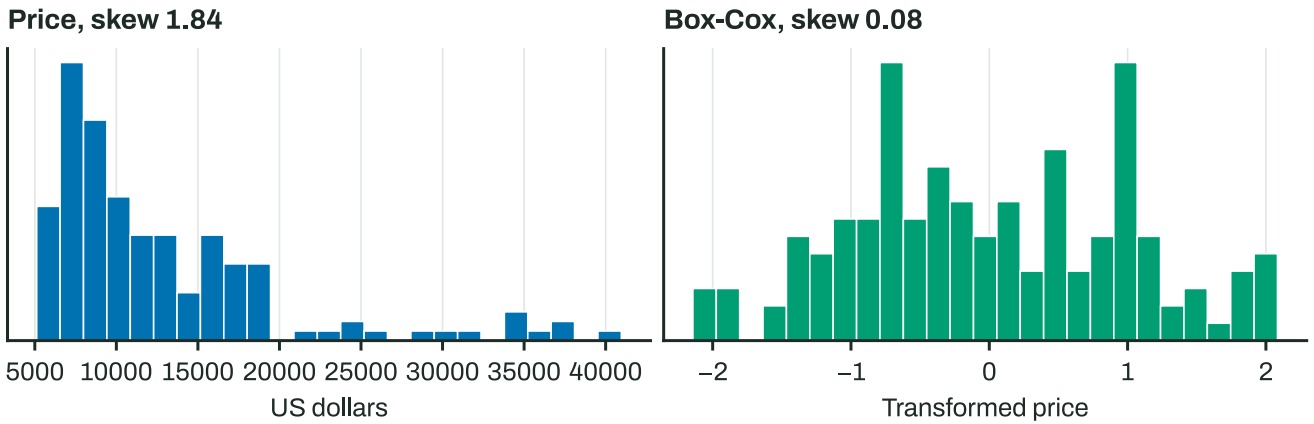
Scikit-learn has no frequency encoder, so a transformer learns the shares in `fit` and maps them in `transform`. The make `renault` occurs only among the test cars, so it gets the share 0.

Three Scalers



The standard scaler subtracts the mean and divides by the standard deviation. Min-max scaling maps the training range to [0, 1] and squeezes the petrol cars into its bottom fifth. The robust scaler uses the median and the interquartile range, so the bulk keeps a usable scale and the diesel cars lie far out.

Power Transforms



PYTHON

```
box_cox = preprocessing.PowerTransformer(
    method="box-cox").fit(y_train.to_frame())
box_cox.lambdas_.round(2)
```

OUTPUT

```
array([-0.66])
```

A scaler keeps the shape of a column, and a power transform changes it. Box-Cox finds the power λ that makes positive values closest to normal, with $\lambda = 0$ meaning the logarithm. Yeo-Johnson also accepts zero and negative values.

NOTEBOOK · SECTIONS 8 TO 10

Outliers, Encoding, and Scaling

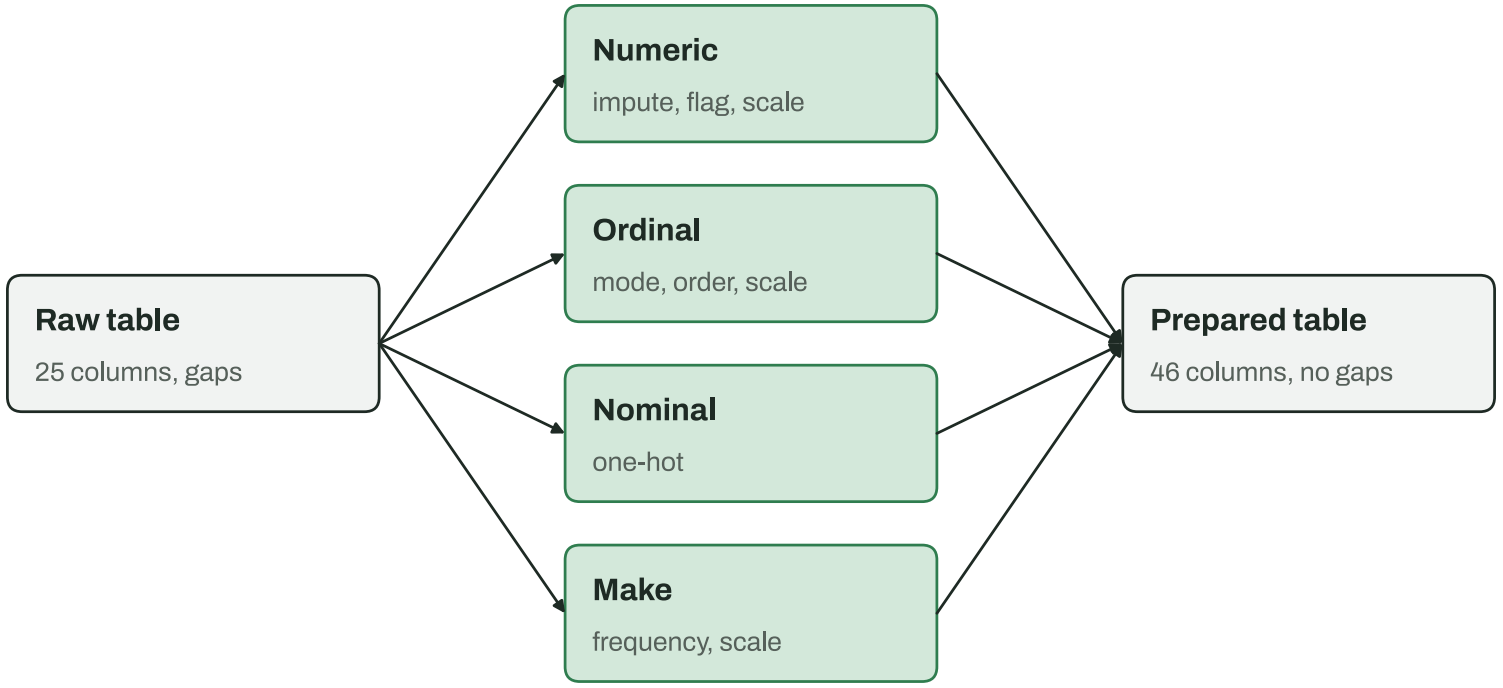
Run the cells from `## 8. Outliers` to `## 10. Scaling and Power Transforms`.

03

The Pipeline and Leakage

One object for every step, fitted on the training rows only

One Pipeline



`ColumnTransformer` applies one chain of steps to each group of columns, and `make_pipeline` chains the steps of a group, so that one call to `fit` learns every statistic from the rows it is given.

The Preparation in Code

PYTHON

```python
preparation = compose.ColumnTransformer([
    ("numeric", make_pipeline(impute.IterativeImputer(max_iter=20, random_state=SEED, add_indicator=True),
                              preprocessing.StandardScaler()), numeric),
    ("ordinal", make_pipeline(impute.SimpleImputer(strategy="most_frequent"),
                              preprocessing.OrdinalEncoder(categories=orders),
                              preprocessing.StandardScaler()), ordinal),
    ("nominal", one_hot, nominal),
    ("make", make_pipeline(FrequencyEncoder(), preprocessing.StandardScaler()), ["make"]),
]).set_output(transform="pandas")
prepared_train = preparation.fit_transform(X_train)
prepared_test = preparation.transform(X_test)
prepared_train.shape, prepared_test.shape, prepared_test.isna().sum().sum()
```

OUTPUT

```
((150, 46), (51, 46), np.int64(0))
```

Reading the Fitted Steps

PYTHON

```python
fitted_numeric = preparation.named_transformers_["numeric"]
names = fitted_numeric[0].get_feature_names_out()
pd.Series(fitted_numeric[-1].mean_, index=names).round(2).tail(3)
```

OUTPUT

```
missingindicator_normalized-losses    0.15
missingindicator_bore                 0.02
missingindicator_stroke               0.02
dtype: float64
```

`named_transformers_` reaches a fitted group, and indexing a pipeline reaches its steps. A flag is 0 or 1 before it is scaled, so its mean is the share of training cars with the gap.

Leakage

PYTHON

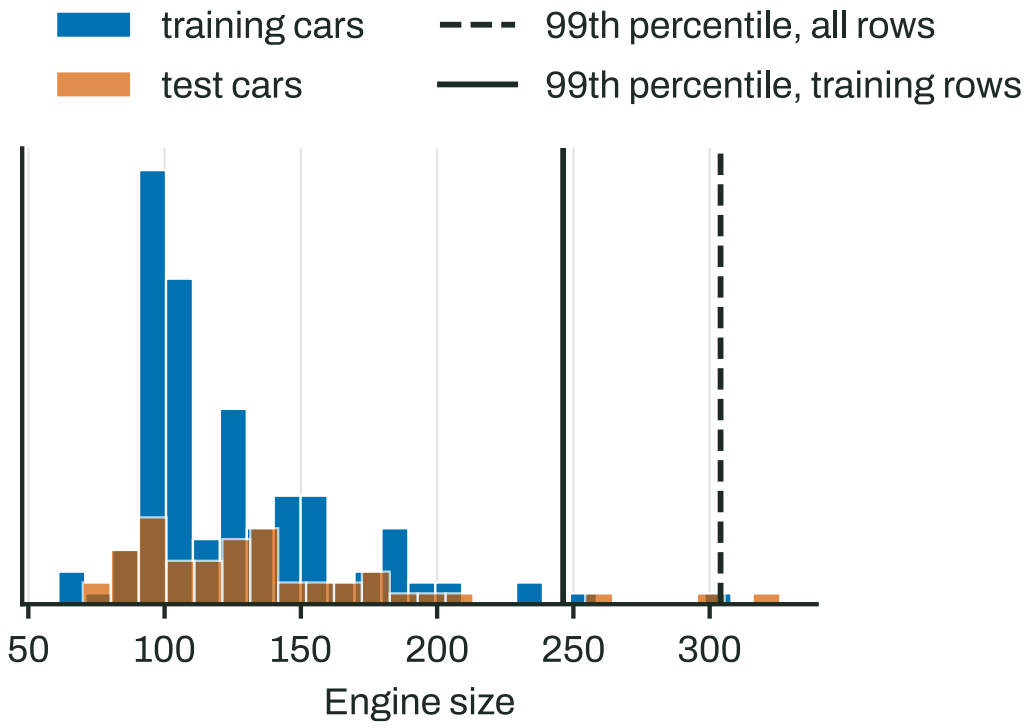
```
leaky = clone(preparation).fit(X)
honest = clone(preparation).fit(X_train)
measured = ["numeric__engine-size", "numeric__curb-weight", "numeric__horsepower"]
pd.DataFrame({"all rows": leaky.transform(X_test)[measured].std(),
              "training rows": honest.transform(X_test)[measured].std()}).round(2)
```

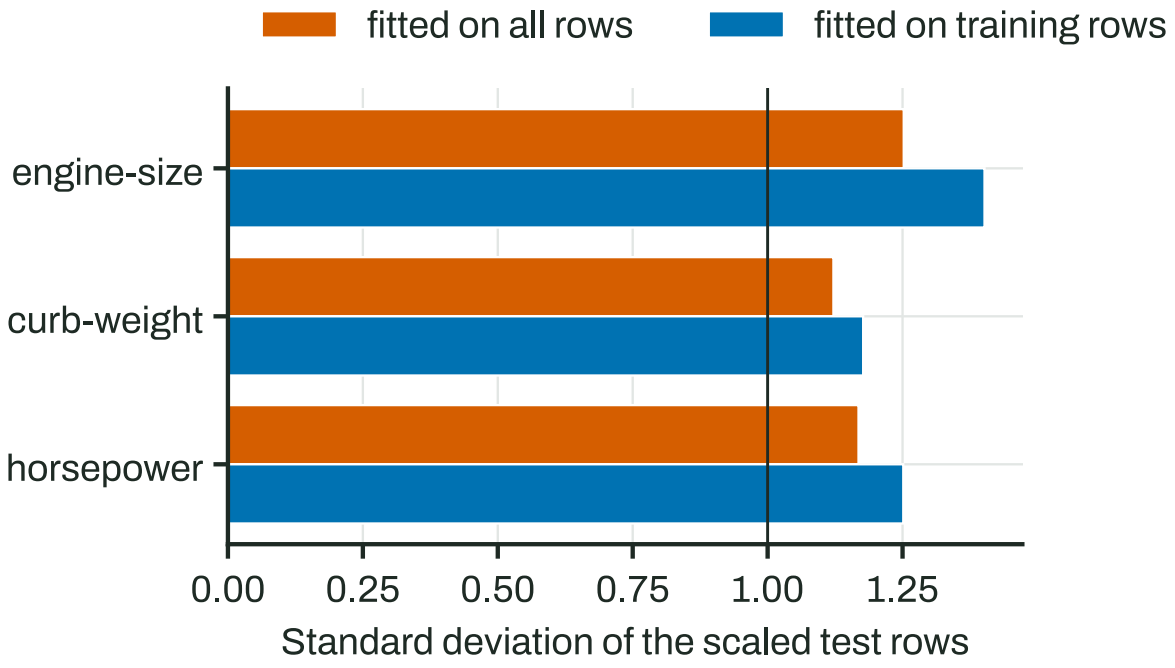
OUTPUT

	all rows	training rows
numeric__engine-size	1.25	1.40
numeric__curb-weight	1.12	1.18
numeric__horsepower	1.17	1.25

Leakage happens when the rows a result is judged on take part in the fitting. Fitted on all rows, the preparation has seen the test cars, so their scaled columns spread less than they would for new cars.

What Leakage Changes





The large test engines raise the 99th percentile from 246 to 304 cubic inches, and the make `renault` gets a share it does not have in the training rows. A pipeline is fitted on the training rows only and transforms the test rows, so held-out rows never shape it.

The Decisions

Decision	Reason
Drop the rows without a `price`.	A car without the response cannot train a model.
Impute `normalized-losses` and flag it.	Its gaps depend on the make.
Impute `bore` and `stroke` and flag them.	A rotary engine has neither.
Encode the counts in words in their order.	An alphabetical code breaks the order.
Encode `make` by its frequency.	Several of its 22 makes have one or two cars.
Fit every step on the training rows only.	Fitted on all rows, the test cars look ordinary.
Keep the outliers.	They are correct values of real cars.
Standardize every column.	Columns in pounds and in inches become comparable.

NOTEBOOK · SECTIONS 11 TO 13

The Pipeline and Leakage

Run the cells from `## 11. One Pipeline` to `## 13. The Prepared Table`.



Reference

Variants, alternatives, and parameters for the tasks of this part

Deletion in pandas

Option	When It Is the Right Choice
`df.dropna(subset=[...])`	It drops the rows with a gap in the named columns, such as the response.
`df.dropna(thresh=n)`	It keeps the rows with at least `n` values present.
`df.dropna(axis=1)`	It drops every column with a gap, when the column is mostly empty and its absence carries nothing.
`df.dropna()`	It drops every row with a gap, when the gaps are few and missing completely at random.

Deletion or Imputation

Situation	Decision
The response is missing.	Drop the row, because it can neither train nor test a model.
A value does not exist, such as the bore of a rotary engine.	Fill a constant or a typical value and add a flag.
A column is mostly empty.	Drop the column, unless its absence is informative.
Dropping rows would remove whole groups.	Impute, because the rows that remain are no longer a fair sample.

Simple Imputers

Option	When It Is the Right Choice
`SimpleImputer(strategy="median")`	It fills a numerical column whose values are skewed or hold outliers.
`SimpleImputer(strategy="mean")`	It fills a roughly symmetric numerical column.
`SimpleImputer(strategy="most_frequent")`	It fills a categorical column, or a count with few values.
`SimpleImputer(strategy="constant", fill_value=)`	It fills a gap that has a meaning, such as 0 for an absent quantity or `"none"` for an absent label.

Model-Based Imputers and Flags

Option	When It Is the Right Choice
`KNNImputer(n_neighbors=5)`	It fills a gap from similar rows, on scaled columns, when the columns are related.
`IterativeImputer(max_iter=, random_state=)`	It predicts each column from the others, when the gaps are missing at random.
`add_indicator=True`, `MissingIndicator()`	They flag the imputed cells, when the absence itself may carry information.

The iterative imputer is experimental in scikit-learn, so it needs `from sklearn.experimental import enable_iterative_imputer` before it is imported.

Imputation by Mechanism

Situation	Decision
Gaps are few and missing completely at random.	Deletion or simple imputation is unbiased.
Gaps depend on observed columns.	Use a model-based imputer, and add a flag.
Gaps depend on the missing value.	Every fill is biased, so flag the gap and state the limitation.

Interpolation in pandas

Option	When It Is the Right Choice
`s.asfreq("h")`	It inserts a row for every missing step of a regular series, so the gaps become visible.
`s.interpolate()`	It draws a straight line between the neighbours, for evenly spaced steps.
`s.interpolate(method="time")`	It weights the line by the time between the neighbours, for an uneven index.

Other Fills of a Series

Option	When It Is the Right Choice
`limit=`, `limit_area="inside"`	They fill only short gaps and only between observed values.
`s.ffill()`, `s.bfill()`	They carry the last or the next value, for a quantity that stays constant until it changes.
`s.fillna(0)`	It fills a gap that means none, such as an hour without rentals.

Outlier Rules

Option	When It Is the Right Choice
The IQR rule, $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$	It suits one skewed column, because quartiles ignore the extremes.
The z-score, $\lvert z \rvert > 3$	It suits one roughly normal column, and extremes inflate its own scale.

Outlier Models and Capping

Option	When It Is the Right Choice
`IsolationForest(contamination=, random_state=)`	It flags rows that are unusual across many columns.
`LocalOutlierFactor(n_neighbors=, contamination=)`	It flags rows in sparse regions next to dense ones.
`LocalOutlierFactor(novelty=True)`	It scores new rows with `predict` after fitting on the training rows.
`s.clip(low, high)`	It caps values at bounds learned from the training rows.

Handling an Outlier

Situation	Decision
The value is an error, such as a typing mistake.	Remove it, or set it to missing and impute it.
The value is correct and another column explains it.	Keep it.
The value is correct but dominates a fit.	Cap it at training bounds, or transform the column.
The column is skewed with a long tail.	Apply a power transform, or the logarithm.

Ordinal Encoding

Option	When It Is the Right Choice
`OrdinalEncoder(categories=[[...]])`	It encodes an ordered variable, with the order passed explicitly.
`OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)`	It gives a category unseen in fitting its own code.

One-Hot and Other Encodings

Option	When It Is the Right Choice
`OneHotEncoder(handle_unknown="ignore")`	It encodes a nominal variable, with an unseen category as zeros.
`OneHotEncoder(drop="if_binary")`, `drop="first"`	They remove a column that the others determine, which an unpenalized linear model needs.
`OneHotEncoder(min_frequency=, max_categories=)`	They group rare categories into one infrequent column.
Frequency encoding	It replaces a category with its share of the training rows, for many rare categories.

Scalers

Option	When It Is the Right Choice
`StandardScaler()`	It gives each column mean 0 and standard deviation 1, the default before a penalty or a distance.
`MinMaxScaler(feature_range=(0, 1))`	It maps a bounded column to a fixed range, without outliers.
`RobustScaler(quantile_range=(25, 75))`	It scales by the median and the IQR, for a column with outliers.

Power Transforms in Detail

Option	When It Is the Right Choice
`PowerTransformer(method="box-cox")`	It makes a strictly positive skewed column closer to normal.
`PowerTransformer(method="yeo-johnson")`	It does the same for a column with zero or negative values, and it is the default.
`standardize=True`	It standardizes the transformed column, which is the default of `PowerTransformer`.

Holding Out Rows

Option	When It Is the Right Choice
`train_test_split(X, y, test_size=0.25)`	It holds out a share of the rows once, before any step is fitted.
`random_state=`	It fixes the shuffle, so the split is the same on every run.
`shuffle=False`	It keeps the order of the rows, for rows ordered in time.

Building a Pipeline

Option	When It Is the Right Choice
`make_pipeline(step, ...)`	It chains steps with generated names.
`Pipeline([("name", step)])`	It chains steps with chosen names, for `set_params`.
`ColumnTransformer([(name, steps, cols)])`	It applies one chain to each group of columns.
`make_column_selector(dtype_include="number")`	It selects a group by type instead of by name.
`remainder="drop"`, `"passthrough"`	It drops the unlisted columns, or keeps them unchanged.

Inspecting a Pipeline

Option	When It Is the Right Choice
`pipe[0]`, `pipe[-1]`, `pipe.named_steps`	They reach a fitted step of a pipeline.
`ct.named_transformers_["name"]`	It reaches a fitted group of a column transformer.
`get_feature_names_out()`	It names the columns the preparation produces.
`set_output(transform="pandas")`	It returns a data frame with those names, and it needs `OneHotEncoder(sparse_output=False)`.

Fitting Without Leakage

Call	Rows It Receives
`fit`, `fit_transform`	The training rows only.
`transform`	The training rows, the test rows, or any new rows.
`clone(step)`	It copies a step unfitted, so the same preparation can be fitted on other rows.

Every statistic a step learns, such as a median, a category list, a scale, or a capping bound, comes from the rows passed to `fit`.